

Loading and Displaying 'lion.jpg'

Now, let's load `/content/lion.jpg` and display it. We'll again convert it from BGR to RGB for correct `matplotlib` display.

✓ Harris Corner Detection

Harris Corner Detection is an interest point detector that identifies corners in an image. A corner can be defined as a point where there is a significant change in intensity in multiple directions (both x and y). The algorithm works by considering a small window around each pixel and calculating the change in intensity for shifts of this window in different directions. It then calculates a 'corner response function' (R) for each pixel. Pixels with a high 'R' value are considered corners.

The key steps are:

1. **Compute Image Gradients:** Calculate the gradients G_x and G_y (e.g., using Sobel or Prewitt operators, which we've already done for Sobel).
2. **Compute Products of Gradients:** Calculate G_x^2 , G_y^2 , and $G_x * G_y$.
3. **Compute Sums of Products in a Window:** Sum these products within a local window (e.g., a Gaussian window) to create a **Structure Tensor** (also known as the Second Moment Matrix).
4. **Compute Corner Response Function (R):** For each pixel, $R = \det(M) - k * (\text{trace}(M))^2$, where M is the structure tensor, $\det(M)$ is its determinant, $\text{trace}(M)$ is its trace, and k is an empirical constant (typically 0.04 to 0.06).
5. **Thresholding and Non-Maximum Suppression:** Apply a threshold to R to identify potential corners, and then use non-maximum suppression to find the local maxima among these potential corners, refining the corner locations.

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

# Ensure gray_image_float is available from previous steps
if 'gray_image_float' not in locals():
    print("Error: 'gray_image_float' not found. Please run the previous cell.")
else:
    # Convert the image to a suitable format for Harris Corner Detector (CV)
    # We already have gray_image_float as np.float32, which is CV_32F

    # Apply Harris Corner Detector
    # blockSize: It is the size of the neighbourhood considered for corner
    # ksize: Aperture parameter for the Sobel operator
    # k: Harris detector free parameter in the equation
```

```
harris_response = cv2.cornerHarris(gray_image_float, blockSize=2, ksize=3)

# Result is dilated for marking the corners, not for actual corner calculation
harris_response = cv2.dilate(harris_response, None)

# Threshold for an optimal value, it may vary depending on the image.
# We'll create a copy of the original image to draw on.
corners_image = lion_rgb_image.copy()

# Pixels above a certain threshold (0.01 * max_response) are marked as corners
corners_image[harris_response > 0.01 * harris_response.max()] = [255, 0, 0]

# Display the original image with detected corners
plt.figure(figsize=(10, 10))
plt.imshow(corners_image)
plt.title('Harris Corners Detected on Lion Image')
plt.axis('off')
plt.show()

print(f"Harris Corner Response min: {harris_response.min():.2f}, max: {harris_response.max():.2f}")
```

Harris Corners Detected on Lion Image



Harris Corner Response min: -2143948.00, max: 40782840.00

✓ Canny Edge Detection

The Canny edge detection algorithm is a multi-stage algorithm to detect a wide range of edges in images. It is known for its ability to detect strong and weak edges while suppressing noise and minimizing false positives.

The main steps of the Canny algorithm are:

1. **Noise Reduction:** The image is smoothed using a Gaussian filter to remove noise that can cause spurious edges.

2. **Gradient Calculation:** The intensity gradients of the image are calculated using a Sobel filter. This provides information about the edge strength (magnitude) and direction.
3. **Non-maximum Suppression:** Only local maxima in the gradient magnitude image are kept, thinning the edges to one-pixel wide lines.
4. **Double Thresholding:** Two thresholds (high and low) are used to classify edge pixels. Pixels with gradient values above the high threshold are marked as strong edges. Pixels between the two thresholds are marked as weak edges. Pixels below the low threshold are suppressed.
5. **Edge Tracking by Hysteresis:** Weak edges are only included if they are connected to strong edges. This process helps to track true edges through the image and eliminate isolated noise pixels.

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

# Ensure gray_image_float is available from previous steps
if 'gray_image_float' not in locals():
    print("Error: 'gray_image_float' not found. Please run the previous cell
else:
    # Canny Edge Detector requires grayscale image (which we have as gray_im
    # The function takes two threshold values. Edges with intensity gradient
    # those below min_val are sure non-edges. Edges between these two thresh
    # based on their connectivity.

    # It's common to cast float32 back to uint8 for Canny, or ensure the ran
    # However, cv2.Canny can work with float32 if the range is 0-255.

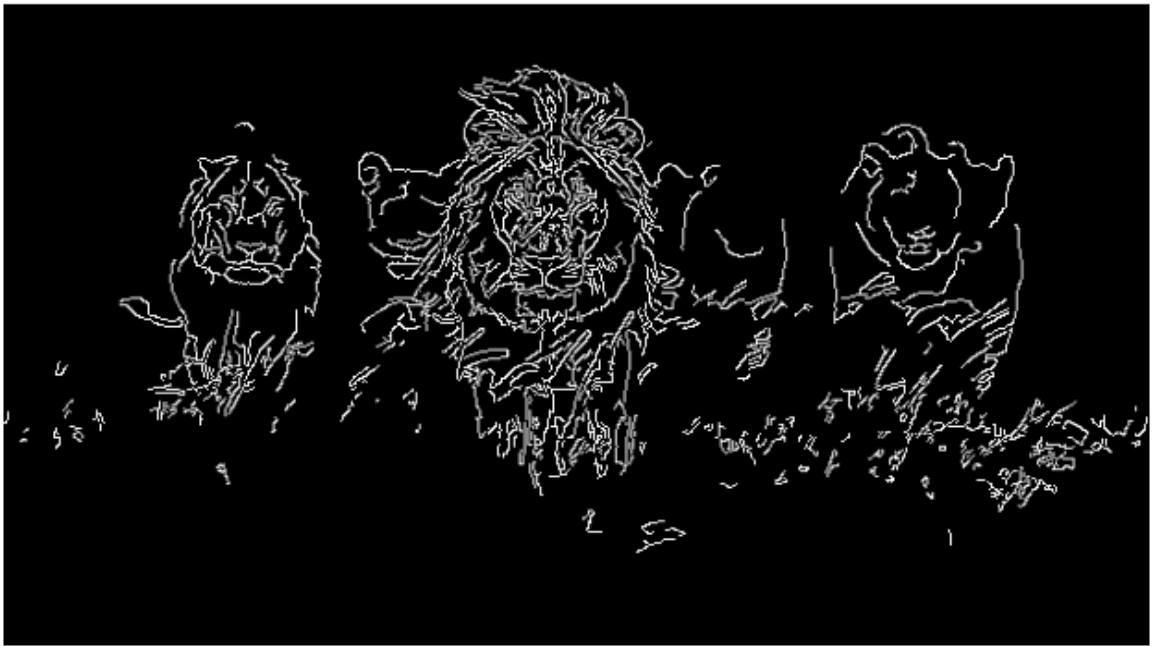
    # We'll use 8-bit unsigned integer version of the grayscale image for Ca
    gray_image_uint8 = np.uint8(gray_image_float)

    # Apply Canny edge detector
    # Recommended ratio for thresholds is 1:2 or 1:3
    edges_canny = cv2.Canny(gray_image_uint8, threshold1=100, threshold2=200

    # Display the Canny edge detection result
    plt.figure(figsize=(8, 8))
    plt.imshow(edges_canny, cmap='gray')
    plt.title('Canny Edge Detection for Lion Image')
    plt.axis('off')
    plt.show()

    print(f"Canny Edge Map (min: {edges_canny.min()}, max: {edges_canny.max(
```

Canny Edge Detection for Lion Image



Canny Edge Map (min: 0, max: 255)